Full-Stack Developer Assessment — Write-up

Candidate: MichaelJohn Revilla **Project:** User Admin (User & Address management) **Date:** 2026-05-27 **Repository:** git@github.com:arukenimon/React-SpringBoot.git

---

1. Overview

A small admin web app that lets administrators **view a list of users and modify their profiles and one-to-many addresses**. The backend exposes a clean REST contract over an in-memory store; the frontend is a focused two-route SPA built around an intuitive "pick a user, then perform all their edits without bouncing between routes" flow.

The brief permitted hardcoded data on the backend, so I chose the contract and the UI flow as the primary surfaces to design with care — those are what the rubric actually evaluates.

---

2. Stack at a glance

Layer	Choice	Why
Backend runtime	Java 17 (works on 26)	Required by the brief
Backend framework	Spring Boot 3.3	De-facto standard, fast to set up, clean DI
Backend storage	`ConcurrentHashMap` in `UserService`	Thread-safe, zero infra; brief said no DB required
Build	Maven 3.9.9 via included `mvnw` wrapper	Zero-install for the reviewer
Frontend	React 18 + TypeScript + Vite	Modern baseline, fast dev loop, type-safety on the contract
UI	MUI v6 (Material UI)	Required by the brief; broad components let me focus on layout & flow
Routing	React Router v6	Standard, declarative
Server state	TanStack Query + axios	Handles loading / error / cache / refetch elegantly
Forms	react-hook-form + Zod	Minimal re-renders + schema-driven validation; one source of truth for form shape

---

3. How to run it

```
# Backend  (terminal 1)
cd backend
```

```
./mvnw spring-boot:run            # mvnw.cmd on Windows cmd / PowerShell
# → http://localhost:8080

# Frontend (terminal 2)
cd frontend
npm install
npm run dev
# → http://localhost:5173
```

Open `http://localhost:5173` in the browser. Vite proxies `/api/*` to the backend, so nothing else needs configuring. The store is seeded on startup with 5 users (Ada Lovelace, Grace Hopper, Alan Turing, Linus Torvalds, Margaret Hamilton) and 1–3 addresses each.

Tests:

```
cd backend && ./mvnw test      # backend unit tests
cd frontend && npm run lint    # TypeScript type-check
```

---

4. Design choices

4.1 API contract — owned hierarchy, two read models

Addresses are **owned** by users; that ownership shows up in the URL:

```
GET    /api/users                                  → UserSummary[]
GET    /api/users/{id}                             → User (full, with addresses)
POST   /api/users
PUT    /api/users/{id}
DELETE /api/users/{id}

GET    /api/users/{id}/addresses                   → Address[]
POST   /api/users/{id}/addresses
PUT    /api/users/{id}/addresses/{addressId}
DELETE /api/users/{id}/addresses/{addressId}
```

Two read models:

- **`UserSummary`** for the list — `id`, `email`, `firstName`, `lastName`, `addressCount`. Lean, lets the table page load fast.
- **`UserDto`** for the detail — includes the full `addresses` array. One round-trip when the detail page mounts.

I deliberately split addresses into a **sub-resource** instead of mutating the whole user document. This gives me granular cache invalidation on the frontend (just the addresses query, not the whole user payload), and avoids the "PATCH a JSON array" anti-pattern.

4.2 In-memory store, behind a service interface

`UserService` owns a `ConcurrentHashMap<UUID, User>`. The controllers depend only on the service, and the service exposes record-based DTOs — never raw domain models. That **DTO boundary** is what makes a future swap to JPA + H2 (or Postgres) a swap rather than a rewrite: the service contract stays the same.

A `@PostConstruct` `DataSeeder` loads ~5 famous-computer-scientist users on boot so the UI

shows realistic data on first load.

4.3 The User → Address flow — one detail page, dialogs for addresses

This is the design choice the brief specifically asks about.

Two routes, period:

- `/users` — MUI `DataGrid` with name, email, and an address-count chip. Each row click navigates to detail.
- `/users/:id` — profile form at top, responsive grid of address cards below, "Add address" button pinned at the section header.

Adding or editing an address opens an MUI `Dialog`, not a new route. The user stays on the same screen they care about — the user they picked — and addresses are batched in mentally as "the things that belong to this person." Real admin workflows are dominated by repetitive edits on one entity; route-bouncing breaks flow and forces you to re-orient on every back-button press.

Delete is gated by a small `ConfirmDialog`; success/failure is announced with a non-blocking `Snackbar`.

4.4 State management — server state vs form state, no Redux

A 1-page admin app does not need Redux. I split state by **lifetime**:

- **Server state** lives in TanStack Query. Custom hooks (`useUsers`, `useUser`, `useUpdateUser`, `useAddresses`, `useAddAddress`, `useUpdateAddress`, `useDeleteAddress`) wrap axios calls. After any address mutation, the addresses query *and* the parent user queries (detail + list) are invalidated — that's what keeps the address-count chip on the list page in sync after you add an address from the detail page.
- **Form state** lives in `react-hook-form`, isolated per form. Zod schemas mirror the backend DTOs, so the form's idea of a valid payload matches what the server will accept.
- **UI state** (dialog open/closed, currently-edited address, toast) is local `useState`.

No Redux, no Context. Both would be ceremony at this scope.

4.5 MUI usage — minimal, themed, responsive

Custom theme with a calm primary, generous shape radius, Inter for type. Components are stock MUI (`AppBar`, `Container`, `Paper`, `Card`, `DataGrid`, `Dialog`, `Snackbar`, `Skeleton`). Address cards live in a `Grid v2` with `{ xs: 12, sm: 6, md: 4 }` breakpoints — they collapse to a single column on phones. Loading states use `Skeleton` (no spinner flash); empty states give a friendly nudge ("No addresses yet — add the first one.").

4.6 Validation — same shape, both sides

Backend uses Bean Validation (`@NotBlank`, `@Email`, `@Size`) on request DTOs. A `@RestControllerAdvice` maps validation failures to `400` with a field-level `details` map, and `NotFoundException` to `404`.

Frontend uses Zod schemas that mirror those constraints. The schema is checked before the request is even sent, so the user sees inline field errors instantly; the backend remains the source of truth on the way in.

---

5. Trade-offs I made deliberately

- **No persistence** — the brief said it wasn't required, and adding JPA/H2 would have eaten time better spent on the UI flow. Mitigated by the DTO boundary, which makes adding it later cheap.
- **No auth** — same reasoning. A real admin tool would need it, but the rubric doesn't.
- **No pagination on the user list** — at 5–10 seeded rows, `DataGrid`'s built-in 10-per-page client-side pagination is enough. If the data grew, I'd switch to server-side paging.
- **A single, focused detail page** rather than separate routes for profile vs addresses — fewer screens, fewer route transitions, the workflow stays close to one click.
- **MUI bundle size** — yes, the production build emits a ~1 MB JS bundle pre-gzip. Code-splitting `DataGrid` and dialogs into separate chunks would trim it; out of scope for this prompt.

---

6. What I'd add with more time

- **Persistence** — swap the `ConcurrentHashMap` for H2 + Spring Data JPA behind the same service interface (~1 hour swap).
- **Optimistic UI** — TanStack Query's `onMutate` for instant feedback on edits.
- **Auth** — even a simple bearer-token guard + a fake login screen.
- **Tests** — `@WebMvcTest` for the controllers, React Testing Library for the dialogs and forms.
- **Pagination & search** on the users list once it grows past one screen.
- **Dockerfile per side** + **`docker-compose.yml`** for one-command spin-up.

---

7. Personal experience callouts

> *MichaelJohn: edit these three bullets before exporting. The grader specifically asks for real examples.*

- **Server-state separation:** *<short anecdote — e.g. "On `<project>` I migrated a Redux-heavy admin to TanStack Query and cut ~400 LOC of boilerplate.">*
- **Form schema reuse:** *<short anecdote — e.g. "Used Zod schemas shared between FE and BE on `<project>` so the contract stayed in sync after refactors.">*

- **Owned-resource API shape:** *<short anecdote — e.g. "Modelled `Account` → `Locations` the same way on `<project>` and avoided the orphan-resource bugs we'd had with flat tables.">*

---

8. Submission

- **Code: or** `Revilla_MichaelJohn_AssessmentForFullStackDeveloper_2026-05-27.zip`
- **This document (PDF):** `Revilla_MichaelJohn_AssessmentForFullStackDeveloper_2026-05-27.pdf`
- **Estimated review time:** ~20 minutes to read + ~5 minutes to boot and click through

Thank you for reviewing.